Package 'magComp'

November 4, 2026

Type Package

Title Bayesian Magnitude-Composition Models

Version 0.0.7

Author Andrew O. Finley [aut, cre],
Arne Nothdurft [ctb],
Arthur Spitta [ctb],
Malcolm S. Itter [ctb]

Maintainer Andrew O. Finley <finleya@msu.edu>

URL https://finleya.github.io/magComp/,
https://github.com/finleya/magComp

BugReports https://github.com/finleya/magComp/issues

Description Fits Bayesian magnitude-composition models for fixed-area stem counts and point-sampling basal area. A negative-binomial magnitude model describes overall abundance, and multinomial allocation distributes it among categories. Models support regression, group, spatial, and space-time effects, composition factors, smoothing across ordered categories, posterior prediction, and area-weighted summaries.

License GPL (>= 3)

Encoding UTF-8

Depends R (>= 4.4.0)

Imports BayesLogit,
graphics,
grDevices,
Matrix,
parallel,
stats,
utils

LinkingTo BayesLogit,
Matrix

Suggests ggplot2,
knitr,
rmarkdown,
testthat (>= 3.0.0)

Config/testthat/edition 3

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.2

Contents

magComp-package . . . 2
fitted.magComp . . . 4
magComp . . . 5
magCompComposition . . . 7
magCompControl . . . 8
magCompCovModels . . . 10
magCompCV . . . 12
magCompData . . . 15
magCompFamilies . . . 17
magCompNotation . . . 18
magCompPrediction . . . 19
magCompPriors . . . 22
magCompSettings . . . 24
magCompSmooth . . . 25
magCompStarting . . . 26
magCompTerms . . . 28
plot.magComp . . . 30
ppcSummary . . . 31
predict.magComp . . . 33
samples . . . 35
spatialDiagnostics . . . 38
standSummary . . . 39
summary.magComp . . . 42
summary.magCompPrediction . . . 43

Index **45**

`magComp-package` *magComp: model fitting, prediction, and summaries*

Description

Bayesian magnitude-composition models for fixed-area stem counts and point-sampling basal area. A negative-binomial model describes the total count, and multinomial allocation distributes that count among categories. Users choose one categorical variable or two in a conditional allocation; neither their names nor their conditioning order is tied to a forestry response. Models can include regression, group effects, spatial effects, and composition factors, with smoothing across ordered categories. Fit either component alone by setting the other formula to `NULL`; the existing nested composition structure remains available without magnitude.

Getting started

The website articles simulate data and illustrate magnitude, composition, and full models using forest inventory examples. Their Quarto sources and HTML previews are in the source repository; the articles are not installed with the package.

Prepare data Use `magCompData` with a `fixedArea` or `pointSampling` design. Each sampling unit has total counts, counts by one category, or counts by two crossed categories. Standalone composition does not require a sampling design.

Specify and fit a model Use `magComp`. For two categories, `magCompComposition` specifies the leading allocation and the conditional child allocation. Formula syntax is described in magCompTerms. Set priors with `magCompPriors`, smoothing with `magCompSmooth`, and MCMC controls with `magCompControl`.

Inspect settings Use `magCompSettings` for resolved priors, parameter constraints, and editable starting values. Use `magCompStarting` to extract starts from a previous fit.

Check the chains Use `summary` and `plot`. Both use `samples` and support individual effects and combined coefficients through their `type` argument. `spatialDiagnostics` summarizes spatial parameters. `samples` extracts all retained parameter draws.

Obtain fitted values and predictions Use `fitted` at observed units and `predict` at new units. Both offer expected quantities or simulated observations. magCompPrediction explains the returned quantities and dimensions.

Summarize and assess Use `summary` on stored prediction draws for means and intervals. Use `standSummary` to aggregate stored expected or observation density draws with represented land-area weights, `ppcSummary` to compare observed and replicate counts, and `magCompCV` for cross-validation predictions.

Call generic functions such as `summary(fit)`, `plot(fit)`, `fitted(fit)`, and `predict(fit, newdata)`; R selects the appropriate method. magCompNotation and magCompPrediction are explanatory help topics, not functions.

Technical document

The Methods and Implementation Details guide gives the model equations, notation, priors, full conditional and Metropolis updates, numerical solvers, and prediction calculations. Its source and PDF are maintained in '`dev/modeling_and_software_details/`' in the source repository. Help pages refer to its section titles so those references remain useful when equation and section numbers change.

See Also

magComp, magCompNotation

`fitted.magComp` *Expected values or replicate observations at fitted units*

Description

Returns expected quantities or replicate observations at the original fitted units, using their original exposures and stored posterior latent effects. Every fitted regression, group, spatial, and composition-factor term is included.

Usage

```
## S3 method for class 'magComp'
fitted(object,
                          type = NULL,
                          target = c("expected", "observation"),
                          summary = c("mean", "samples", "quantile"),
                          probs = c(0.025, 0.5, 0.975), ...)
```

Arguments

`object`	A fitted `magComp` object.
`type`	Quantities to return; see magCompPrediction for supported choices. With `NULL`, defaults depend on the fitted components.
`target`	Use `"expected"` for conditional means or `"observation"` to simulate replicate observations from the fitted likelihood.
`summary`	Return posterior means, aligned retained samples, or quantiles. Means and quantiles pool all retained draws across chains.
`probs`	One or more probabilities strictly between zero and one, used for `summary = "quantile"`.
`...`	Must be empty. Unknown arguments are rejected.

Details

`target = "expected"` constructs quantities from the stored parameter and latent-effect draws, without additional random simulation. For the observation target, a full model draws an NB total followed by multinomial allocation. With only magnitude, replicates draw just the total. With only composition, replicates allocate the observed total among categories with no abundance simulation. Expected composition-only category counts are that observed total times the fitted probabilities. Conditional densities additionally require the original area or BAF information. `ppcSummary` compares replicates with observed counts. Both targets support `fixedAreaStems` and `pointBasalArea`. Point-sampling replicates are in-tree tallies; observation magnitudes and category densities multiply those tallies by each point's BAF to give sampled basal-area estimates.

See magCompPrediction for the four returned quantities, their array dimensions, and the treatment of zero totals. New covariates or exposures belong in `predict`; `fitted()` always uses the original fitted units. Observation simulation uses R's current random-number state; use `set.seed()` externally for reproducibility.

Value

A `magCompPrediction` named list with the requested quantity components and `metadata`, even when only one quantity is requested. Access counts with `result$count`. See that help page for array dimensions and metadata fields.

See Also

`summary` for summarizing stored draws; `predict`, `ppcSummary`, `standSummary`

`magComp` *Fit a magnitude-composition model*

Description

Fits a Bayesian model in which a scalar magnitude describes overall density and a composition allocates that magnitude among categories. For fixed-area stems, plot counts follow a negative-binomial total with multinomial allocation. For point sampling, the count model describes in-tree tallies and the magnitude is basal area per unit area.

Usage

```
magComp(data, family = fixedAreaStems(), magnitude = ~1,
        composition = ~1, smooth = magCompSmooth(),
        priors = magCompPriors(), starting = list(),
        control = magCompControl(), tuning = list(), ...)
```

Arguments

`data`	A `magCompData` object, or a data frame with its preparation arguments supplied through `...`.
`family`	A response family: `fixedAreaStems` (the default) or `pointBasalArea`.
`magnitude`	One-sided formula for the scalar magnitude component. Use `NULL` to fit composition only. See magCompTerms for group and spatial terms.
`composition`	A one-sided formula for a single-category response, or a `magCompComposition` specification with one formula per allocation block. Use `NULL` to fit magnitude only.
`smooth`	Composition smoothing from `magCompSmooth`.
`priors`	Prior settings from `magCompPriors`.

starting	Partial or complete starting-value list. Omitted values receive automatic starts. Use the starting template from magCompSettings; see magCompStarting for its structure.
control	MCMC settings from magCompControl. Sampling is the default. To inspect settings without sampling, use run_sampler = FALSE.
tuning	Optional covariance proposal standard deviations, organized like starting with a covariance list in each spatial term. Values are on the bounded-logit scale; default 0.1. See magCompCovModels.
...	Arguments passed to magCompData when data is a data frame.

Details

Explicit NULL disables a component; omitting either formula still uses its intercept-only default ~1. At least one component is required. With composition = NULL, fit total counts without category labels or sum the supplied category counts. With magnitude = NULL, fit category allocation conditional on observed totals. Composition may still have one categorical response or the existing parent/child structure, with different covariates, factors, and smoothing in each block. Composition-only fitting requires integer category counts, not proportions, and does not require areas.

Only active components are initialized, sampled, and reported in parameter settings, starts, samples, and diagnostics. Nondefault priors or starting values for disabled components are rejected. Smoothing is unused and must remain at its default when composition is disabled. fitted, predict, magCompCV, and ppcSummary support each configuration; standSummary aggregates their density or magnitude samples using explicit represented land areas.

Both components support regression effects. Magnitude formulas can add iid, GP, or NNGP effects; composition formulas can add corresponding factor terms. Colon interactions allow covariates to multiply latent effects. See magCompTerms for syntax and identification constraints.

The composition_link control selects probit (the default) or logit. The selected link applies to all allocation blocks, fitted values, and predictions. Ordered composition coefficients can use RW1 or RW2 smoothing.

Each magnitude process term has an estimated variance. Composition factors have unit variance, with positive estimated anchor-diagonal loadings and zeros above the anchor diagonal. Covariance parameters are estimated unless their starting entry uses fixed(value); see magCompCovModels for family-specific defaults. Negative-binomial size is estimated by default under a Gamma(shape = 1, rate = 0.01) prior when the magnitude formula has no latent process terms. With magnitude iid or spatial terms, size is fixed at control$nb_size (default 1000). See magCompPriors to change the prior or explicitly fix size in a regression-only magnitude model.

Use magCompSettings to inspect the resolved model, priors, and starts. The technical guide linked from magComp-package gives the model and sampler details; magCompNotation connects its symbols to software parameter names.

Value

An object of class magComp. Use samples for posterior parameter arrays, summary for parameter summaries, and fitted or predict for response quantities.

See Also

magComp-package, magCompComposition, magCompControl

Examples

```
rows <- expand.grid(plot = paste0("p", 1:8),
                    dbh_class = c("small", "large"))
rows$count <- c(8, 5, 9, 6, 3, 7, 4, 5, 2, 4, 1, 3, 6, 2, 5, 4)
rows$area <- 0.05
rows$height <- rep(seq(-1, 1, length.out = 8), 2)
dat <- magCompData(rows, unit = plot, category = dbh_class,
                   tree_count = count, design = fixedArea(area))

# A short run to illustrate the interface, not assess convergence.
set.seed(1)
fit <- magComp(dat, magnitude = ~ height, composition = ~ height,
  control = magCompControl(n_samples = 60, n_burn = 20, verbose = FALSE))
summary(fit)
fitted(fit, type = "density")$density
predict(fit, data.frame(height = c(-0.5, 0.5)), area = 0.05)$count
```

---

magCompComposition *Specify composition allocation blocks*

Description

Creates a composition specification with one formula per allocation block. For two categories, the first formula allocates among parent categories and the second allocates among child categories within each parent.

Usage

```
magCompComposition(...)
```

Arguments

`...`	Two-sided formulas naming the response category on the left. Conditional blocks use `child	parent ~ ...`.

Details

The category variables are chosen by the user and need not be species or diameter classes. Either the parent or child can be ordered or unordered. Smoothing uses the ordering recorded by `magCompData()`, not the category's name or position.

For example, use `species ~ height` and `dbh_class | species ~ height + cover`. The two blocks may use different covariates and different factor terms. Their category names must match those supplied to magCompData, in parent-then-child order. To reverse the conditioning, reverse the columns

in `categories` and specify the corresponding formulas. This changes the model specification; structured models and their priors need not give equivalent fits in both orders.

A leading block with J_A categories has $J_A - 1$ coefficient rows. A conditional block with J_B child categories has $J_A(J_B - 1)$ rows: a separate child allocation for each parent. Within a conditional block, factor scores are shared across parents, with separate loading rows for each parent/child stick. Each block can have its own factor rank and anchors.

Right-hand sides use ordinary regression formulas and the markers described in magCompTerms. For a single category, a two-sided formula such as `dbh_class ~ height` is equivalent to supplying `composition = ~ height` to `magComp()`.

Value

A `magCompComposition` object to pass to `magComp()`.

See Also

`magComp`, `magCompSmooth`; the technical guide's sections on composition in magComp-package.

Examples

```
magCompComposition(
  species ~ height + factor_nngp(x, y, K = 1),
  dbh_class | species ~ height + cover
)
```

`magCompControl` *MCMC control for magComp*

Description

Creates a control list for `magComp()`. Negative-binomial settings apply only when magnitude is fitted; `composition_link` applies only when composition is fitted.

Usage

```
magCompControl(n_samples = 2000, n_burn = 1000, n_thin = 1,
               n_chains = 1, n_cores = 1,
               chain_seeds = NULL,
               n_report = 100, nb_size = 1000,
               nb_size_tuning = 0.25,
               metropolis = list(),
               composition_link = c("probit", "logit"),
               run_sampler = TRUE, verbose = TRUE)
```

Arguments

n_samples	Total number of MCMC iterations per chain, including burn-in. Must be an integer greater than n_burn.
n_burn	Number of initial iterations discarded from each chain.
n_thin	Thinning interval after burn-in. Must leave at least one retained draw per chain.
n_chains	Number of MCMC chains to run from the R side. Chains are run sequentially unless n_cores > 1.
n_cores	Number of R worker processes used for chain-level parallelism on Unix-like systems. Values greater than one fall back to serial execution on platforms without forked processes.
chain_seeds	Optional integer vector of length n_chains. When supplied, each chain is started after setting the corresponding R seed. With NULL, serial fitting uses R's current RNG state, so an external set.seed() can reproduce a fitting and prediction sequence. Supply explicit chain seeds to reproduce parallel fitting under R's default RNG settings.
n_report	Progress reporting interval.
nb_size	Fixed negative-binomial size, or its automatic starting value when size is estimated. By default size is estimated for magnitude formulas without iid or spatial terms and fixed otherwise. Set magCompPriors(magnitude = list(size = NULL)) to fix size explicitly; setting nb_size alone does not change estimation status. Large fixed values approximate Poisson magnitude counts.
nb_size_tuning	Initial random-walk Metropolis tuning standard deviation on the log size scale when nb_size is estimated.
metropolis	List with adapt (default TRUE), batch_length (default 25), and target_accept (default zero, selecting 0.44 for scalar proposals and 0.234 for multivariate proposals). An explicit value between zero and one overrides the target. Covariance blocks use an initial scale search within burn-in, followed by diminishing adaptation of proposal covariance and scale, including after burn-in. The separate size update adapts its scalar scale only during burn-in.
composition_link	Composition stick-breaking link: "probit" (the default) or "logit". The same link applies to all allocation blocks.
run_sampler	Logical; TRUE (the default) fits the model. FALSE prepares the model and resolves settings without sampling.
verbose	Logical; print sampler progress.

Details

Each chain runs exactly n_samples iterations. The number of retained draws is floor((n_samples - n_burn) / n_thin). The first saved draw is at iteration n_burn + n_thin. Any final iterations after the last thinning boundary still run but are not saved. For example, 25,000 total iterations, 10,000 burn-in iterations, and thinning by 10 retain 1,500 draws per chain. Output array dimensions and prediction metadata count retained draws, not total iterations.

The initial covariance scale search uses at most 20 complete batches and never exceeds burn-in. Each batch contains full sampler sweeps. After that search, adaptation uses gain $(b + 1)^{-0.8}$ for

adaptive batch number b; the empirical covariance uses the chain states, including rejected proposals. There is one block per magnitude process and one per composition factor. Fixed covariance parameters are excluded. Set `adapt = FALSE` to use the initial proposal throughout; with no burn-in the initial search is skipped.

The composition link applies to all allocation blocks in a fit. Fitted values and prediction use the matching inverse link when converting composition linear predictors to stick-breaking probabilities.

Value

A `magCompControl` object.

See Also

`magCompPriors`, `summary`; the technical guide's Sampler algorithm and Numerical implementation sections in magComp-package.

Examples

```
magCompControl(n_samples = 25000, n_burn = 10000, n_thin = 10)
```

---

magCompCovModels *Covariance models and their parameter settings*

Description

Lists the covariance families, parameter names, and domains supported by GP and NNGP terms. The same covariance calculations are used for fitting and prediction. `uniform()` specifies a prior; `fixed()` marks a starting value as fixed throughout fitting.

Usage

```
magCompCovModels()
uniform(lower, upper)
fixed(value)
```

Arguments

`lower`, `upper`	Finite bounds for a uniform covariance-parameter prior, with `lower` < `upper`. Bounds must respect the parameter's domain.
`value`	A fixed parameter value, or a named vector of values for factors.

Details

Specify a family with `cov_model` inside `gp()`, `nngp()`, `factor_gp()`, or `factor_nngp()`. The default is `"exp"`. For space-time families, the final coordinate is numeric time, for example `nngp(easting, northing, year, cov_model = "sep_exp", m = 15)`. Time gaps are used as supplied. Let h be spatial distance, u absolute time separation, and d the number of spatial coordinates. Correlations are:

`exp` $\exp(-\phi h)$, with `phi > 0`.

`matern` $(\phi h)^{\nu} K_{\nu}(\phi h)/(2^{\nu-1}\Gamma(\nu))$, with `phi > 0` and `nu > 0`. At `nu = 0.5` this is exponential.

`sep_exp` $\exp(-\phi h - \lambda u)$, with positive `phi` and `lambda`.

`multi_res_sep_exp` $\alpha \exp(-\phi_1 h - \lambda_1 u) + (1-\alpha)\exp(-\phi_2 h - \lambda_2 u)$. The mixing weight `alpha` is between zero and one; all four decays are positive.

`gneiting` Write $T = 1 + a u^{2\alpha}$. Correlation is $T^{-\delta - d/2}\exp(-c h^{2\gamma}/T^{\beta\gamma})$. Parameters `a` and `c` are positive; `alpha` and `gamma` are in $(0, 1]$; `beta` is in $[0, 1]$; `delta` is nonnegative.

Magnitude process variance is separate from these correlations and has an inverse-gamma Gibbs update. Composition factors have unit marginal variance.

All estimated covariance parameters have independent bounded uniform priors. Put overrides in a term's `covariance` list under `priors`, `starting`, or `tuning`. The same parameter names are used in `samples()` and `summary()`. For composition, a common scalar setting applies to all factors; a named vector of starts/tuning values or a named list of priors can override individual factors. A named list of starting values can mix `fixed(value)` and ordinary numeric values across factors. Omitted entries retain defaults. `magCompSettings()` shows the resolved supports and starts. Fixed parameters do not have priors or proposals.

By default, Matérn smoothness has prior `uniform(0.05, 2)`. Gneiting `alpha = fixed(1)`, `gamma = fixed(0.5)`, and `delta = fixed(0)`; `a`, `c`, and `beta` are estimated. An ordinary starting value or a prior override releases a default-fixed parameter.

Other default supports are calibrated to spatial distances and the time span. For the mixture, the default supports of `phi_1` and `phi_2` meet at $30/D$, where D is maximum spatial separation; component 1 describes longer spatial dependence. Temporal supports are calibrated to dependence up to twice the observed time span. User overrides are direct parameter bounds, not range parameters. Overlapping mixture supports can permit label switching. When a coordinate domain has no positive span, supply the corresponding support or fix its parameter. The technical guide gives the default formulas.

Covariance parameters are proposed jointly within each magnitude process and each composition factor, after bounded-logit transformation, with Jacobian correction. The `tuning` entries are initial transformed-scale standard deviations (default 0.1). `magCompControl(metropolis = )` controls adaptation.

Value

`magCompCovModels()` returns a named list of registry entries. `uniform()` returns a prior specification and `fixed()` a marked numeric value for a covariance starting-value list.

See Also

magCompTerms, magCompPriors, magCompStarting, magCompSettings, magCompControl

Examples

```
magCompCovModels()$sep_exp
~ nngp(easting, northing, year, cov_model = "sep_exp", name = "space_time")
magCompPriors(magnitude = list(terms = list(space_time = list(
  covariance = list(phi = uniform(0.1, 4), lambda = uniform(0.02, 2))))))
# A matching starting-value list for magComp(starting = ...).
list(magnitude = list(terms = list(space_time = list(
  covariance = list(phi = 1, lambda = fixed(0.3))))))
```

---

magCompCV *Cross-validation at held-out sampling units*

Description

Fits a magComp model repeatedly while holding out complete sampling units, retaining the original category levels. Each fold predicts all held-out units together through predict, using their covariates, group labels, coordinates, and original sampling exposures. Prediction options have the same defaults and meanings as predict().

Usage

```
magCompCV(data, family = fixedAreaStems(), magnitude = ~1,
  composition = ~1, smooth = magCompSmooth(), priors = magCompPriors(),
  starting = list(), control = magCompControl(), tuning = list(), n_fold = 10,
  fold_id = NULL,
  type = NULL,
  target = c("expected", "observation"),
  summary = c("mean", "samples", "quantile"),
  probs = c(0.025, 0.5, 0.975), keep_fits = FALSE)

## S3 method for class 'magCompCV'
plot(x, type = x$type[1L],
  level = c("cell", "total"), file = NULL, main = NULL, ...)
```

Arguments

data	A data object returned by magCompData.
family	A model family: fixedAreaStems or pointBasalArea. Both support expected and observation targets.
magnitude	Magnitude formula, or NULL to fit composition only.
composition	Composition model formula or magCompComposition specification passed to magComp, or NULL to fit magnitude only.
smooth	Composition smoothing specification.
priors	Prior specification.

starting	Partial or complete starting values specified for the full data. Supplied magnitude effects and composition scores are matched to the full-data template and subset to each training fold's groups or locations. Unspecified starts are computed using only that fold's training data. Supplied loadings must satisfy each fold's anchor constraints; use explicit anchors when reusing full-data loading starts.
control	MCMC control passed to each fold fit. Sampling is enabled.
tuning	Covariance proposal settings passed to each fold fit; see magCompCovModels.
n_fold	Number of folds when fold_id is not supplied. Use n_fold = data$n_unit for leave-one-out cross-validation.
fold_id	Optional vector of fold labels named by sampling unit, identifying at least two folds.
type	A nonempty, unique subset of "magnitude", "proportion", "density", and "count", restricted to the quantities supported by the fitted components. Default NULL selects the same quantities as predict. For plotting, select one returned quantity; the first requested quantity is the default.
target	"expected" returns expected quantities; "observation" simulates observations using the fitted likelihood. Meanings and defaults match predict.
summary	Return means, aligned retained samples, or quantiles, with the same dimensions and behavior as predict.
probs	One or more probabilities strictly between zero and one, used for summary = "quantile". For example, probs = 0.5 requests only the median. Also used for plot intervals when samples were retained.
keep_fits	Logical; if TRUE, retain each fold's fitted magComp object.
x	A magCompCV object.
level	Plot category values ("cell") or within-unit totals across categories ("total"). Magnitude already represents overall density and has the same plot at either level. Proportions support only "cell". Composition-only CV has no total-level plot because unit totals are conditioned on.
file	Optional PNG output path.
main	Optional plot title.
...	Additional graphical arguments passed to plot().

Details

Standalone components are supported. Magnitude-only CV compares total counts or overall densities. Composition-only CV predicts category probabilities; conditional expected counts and simulated allocations use held-out observed totals. These assess allocation given the total, not abundance prediction. The result's count_conditioning is "observed_total" in this case, otherwise "none". Conditional count/density totals and total-level plots are omitted because totals are fixed by construction. Composition-only probability or count predictions need no areas; conditional densities do.

Folds are fitted sequentially. control$n_cores parallelizes chains within a fit, not folds. With no supplied fold_id, units are randomly assigned to approximately equal-sized folds.

Each fold uses one joint prediction call, including every fitted model term. All requested quantities and within-unit totals use the same draws. Totals are summed across categories before taking means or quantiles. Fold identities are retained: posterior draws from separately fitted folds do not constitute joint draws across all held-out units.

Observed counterparts are category counts, category counts divided by exposure (density), total count divided by exposure (magnitude), and category count fractions (proportion). Observed proportions are NA for zero observed totals. For fixed-area stems, exposure is plot area; for point basal area it is $1/\mathrm{BAF}$. CV supplies each held-out unit's original exposure; there is no user-supplied replacement `area` or `newdata` argument.

The definitions and zero-total handling in magCompPrediction apply to all returned predictions.

Expected-target intervals describe uncertainty in expected quantities. Observation-target intervals also include observation variation. CV returns predictions and observed values; it does not calculate predictive scores or coverage statistics.

Plots use means for mean or sample results, and the requested quantile nearest 0.5 for quantile results, with the selected statistic labeled on the axis. Intervals use the smallest and largest requested probabilities when at least two are supplied. Mean-only results have no interval bars. Undefined observed or predicted proportions do not produce plotted pairs.

Serial workflows can be reproduced with external `set.seed()`. Parallel fitting uses `chain_seeds` in `magCompControl()`.

Value

An object of class `magCompCV`, containing:

`predictions` A list named by fold label, each entry a standard magCompPrediction result with the requested quantities, summary, row labels, and prediction metadata. A single requested type still returns that named result.

`observed` A list named by requested type, with matrices in the original sampling-unit order. Category quantities have one column per category; magnitude has one column named `"total"`. Match prediction rows using each fold result's `metadata$unit_ids`.

`totals` A list named by fold label, each containing within-unit category sums for requested `count` and `density` quantities. Each total uses magnitude-style dimensions: unit by sample (and chain) for samples, unit by one column for means, or unit by probability for quantiles. Magnitude is already overall density; no total proportion is returned. Entries are empty lists for composition-only models or if neither count nor density is requested.

`count_conditioning` `"observed_total"` for composition-only assessment, otherwise `"none"`.

`fold_id` Fold assignments named by original sampling unit.

`fits` Fold models named by fold label, or `NULL` unless `keep_fits = TRUE`.

`call`, `target`, `type`, `summary`, `probs` The call and selected prediction options.

See Also

predict, magCompPrediction

Examples

```
rows <- expand.grid(plot = paste0("p", 1:6), cls = c("small", "large"))
rows$count <- rep(c(1, 3, 2, 4, 2, 1), 2)
rows$area <- rep(seq(0.05, 0.1, length.out = 6), 2)
dat <- magCompData(rows, unit = plot, category = cls, tree_count = count,
                   design = fixedArea(area))
set.seed(42)
cv <- magCompCV(dat, n_fold = 2, target = "observation", type = "count",
                summary = "samples",
                control = magCompControl(n_samples = 6, n_burn = 2,
                                         verbose = FALSE))
cv$predictions[["1"]]$count
cv$observed$count
cv$totals[["1"]]$count
```

`magCompData` *Prepare data for magnitude-composition models*

Description

Builds sampling-unit summaries, count matrices, allocation-block summaries, coordinate matrices, design metadata, fixed-area exposure summaries, and expanded tree-list attributes used by `magComp()`. A one-category response creates one marginal allocation block. A two-category response creates a leading marginal block and a conditional child block. Category ordering metadata controls whether RW1/RW2 smoothing can be used for an allocation block. Coordinates must be constant within each sampling unit. When a model is prepared, the same check is applied to its predictors and grouping columns. Tree-level responses and attributes may vary within a unit.

Usage

```
magCompData(data, unit, category = NULL, categories = NULL,
            coords = NULL, tree_factor = NULL, tree_count = NULL,
            design = NULL, attributes = NULL, ordered = NULL)

fixedArea(area = NULL)

pointSampling(baf = NULL)
```

Arguments

`data`	Data frame with one row per observed tree or one row per category tally.
`unit`	Sampling-unit identifier column, supplied as a bare name or a string.
`category`	Optional single category column. Omit categories for total-count magnitude-only data.
`categories`	Optional one- or two-column category specification, supplied as `c(parent, child)` or `c("parent", "child")`. Use this instead of `category` for two-way composition tables.

coords	Optional coordinate columns, usually supplied as c(x,y).
tree_factor	Optional tree expansion factor column retained for expanded attribute summaries and future weighted-response families. Current point-sampling basal-area fits use pointSampling(baf = ...) rather than a tree-factor likelihood.
tree_count	Optional integer tally-count column. When supplied, category count matrices are built from this column rather than assuming one observed tree per row. Zero-count rows can be used to retain sampling units with no observed trees in the modeled categories.
design	Sampling design helper, e.g., fixedArea(area).
attributes	Optional tree-level attributes to expand by tree_factor. These summaries are stored with the prepared data for diagnostics and future continuous-response families.
ordered	Optional logical vector indicating which category columns are ordered. Names should match category column names. Ordered factor columns are also detected automatically. An ordered category must be a factor with levels in the intended order, or numeric (sorted numerically). Character labels require explicit factor levels to avoid accidental alphabetical ordering.
area	Fixed plot area, supplied as a scalar, a vector named by sampling-unit ID, or a column name constant within each unit. Non-scalar vectors must be named; values are matched by unit ID rather than position.
baf	Basal area factor, supplied as a scalar, a vector named by sampling-unit ID, or a column name constant within each unit. Non-scalar vectors must be named; values are matched by unit ID rather than position.

Details

Category columns can describe any categorical response dimensions; species and diameter classes are examples. With two columns, their order in categories determines the parent and child. Either can be ordered or unordered. Supply any desired classes as category values before preparing the data; the constructor does not create classes from continuous measurements.

Without category columns, counts are summed by sampling unit into one column named "total". Supply tree_count for tallied totals or use one row per tree. Fit these data using composition = NULL. Composition-only data still require integer category counts but can omit design; provide area or BAF information if conditional density outputs are needed later.

Column specifications accept bare names or strings. Sampling units retain their order of first appearance. Factor levels determine category order; otherwise numeric categories are sorted numerically and character categories alphabetically. Use explicit factor levels when a particular category order is needed, and ordered factors for categories to be smoothed. Unobserved category cells have zero counts; unused factor levels are retained.

To retain an empty sampling unit, provide a row with tree_count = 0 and valid category labels, covariates, and design information. Without a tally column, every row counts as one tree.

A fixed-area count model requires one common sampling area across all categories within each unit. Nested subplots with different areas for different categories must first be put on a common sampling support. Tree expansion factors do not convert those unequal-area counts into a valid common-area likelihood.

Tree-level attributes are stored as sums of `tree_factor * attribute` by unit and category. They are not responses in the implemented count models. Basal-area fits use in-tree tallies with a `pointSampling()` design and the `pointBasalArea()` family.

Value

The data constructor returns a `magCompData` object. Both design helpers return sampling-design objects.

See Also

`magComp`, `magCompComposition`, `fixedAreaStems`, `pointBasalArea`

Examples

```
rows <- data.frame(plot = c("p1", "p1", "p2"),
                   dbh_class = c("small", "large", "small"),
                   count = c(5, 2, 0), area = c(0.05, 0.05, 0.08))
rows$dbh_class <- ordered(rows$dbh_class, levels = c("small", "large"))
dat <- magCompData(rows, unit = plot, category = dbh_class,
                   tree_count = count, design = fixedArea(area))
dat$counts
```

---

magCompFamilies *Response families for stem counts and basal area*

Description

Constructs a response family for `magComp`. Both families model negative-binomial totals and multinomial category allocation. Either component can be fitted alone by setting the other formula to `NULL`.

Usage

```
fixedAreaStems()

pointBasalArea()
```

Details

`fixedAreaStems()` uses fixed-area plot counts. Its magnitude is stem density M_i, so the expected plot total is $a_i M_i$ for area a_i. Supply a `fixedArea` design when preparing data.

`pointBasalArea()` uses horizontal point-sampling in-tree tallies. Its magnitude is basal area per unit area M_i^{BA}, so the expected tally is M_i^{BA}/BAF_i. Supply a `pointSampling` design. Tree-factor-expanded stem counts or other attributes are not fitted responses.

Negative-binomial size is positive and finite. Both families estimate it by default under a Gamma(shape = 1, rate = 0.01) prior when the magnitude formula has no latent process terms. With magnitude iid or spatial terms, size is fixed at `control$nb_size` (default 1000). Use `magCompPriors` to change

the prior or explicitly fix size. Large sizes approximate a Poisson model; the sampler does not fit an exact Poisson model.

Both families support expected fitted values and predictions, as well as replicate or new observations from the fitted likelihood. Point-sampling count outputs are in-tree tallies, with mean M_i^{BA}/BAF_i; observation magnitude multiplies the tally by BAF to give a sampled basal-area estimate. Composition-only models simulate allocations conditional on observed or supplied totals under either family, including for posterior predictive checks. Plot area or BAF is unnecessary for composition-only fitting and probabilities; it is needed for conditional density. `standSummary` aggregates density or magnitude samples under either family using represented land areas.

Value

A `magCompFamily` object with `likelihood` equal to `"negative_binomial_multinomial"`.

See Also

`magCompData`, `magCompControl`, `magCompPriors`; the technical guide's Fixed-area count model and Point-sampling basal area sections in magComp-package.

`magCompNotation` *Model notation and software parameter names*

Description

Connects the mathematical notation used in the manuscript and technical document to the package's prior settings, starting values, and sample labels. This is a help topic, not a function. The technical guide is linked from magComp-package.

Details

The magnitude model uses regression coefficients β, iid group effects δ_g, and spatial effects $v(s)$. Composition uses intercepts α_{0j} and slope vectors α_j. Iid factor scores u_h have loadings γ_j; spatial factor scores $w(s)$ have loadings λ_j. Additional terms and allocation blocks have their own parameter indices. Both factor types retain unit scale and the specified loading-anchor constraints. The index ℓ denotes a retained posterior draw in the technical document.

The descriptive software names map to those symbols as follows:

Magnitude coefficients β Within `starting$magnitude`, `coefficients` is a vector named by design columns, including `"(Intercept)"`. In the magnitude priors, `coefficient_variance` is σ^2_β.

Composition coefficients α_{0j}, α_j Each composition block has a `coefficients` matrix with one row per coefficient stick and one column per design column. Its intercept column contains α_{0j}. Block prior settings `intercept_variance` and `slope_variance` are $\sigma^2_{\alpha,0}$ and σ^2_α, with defaults inherited from composition settings.

Magnitude effects $\delta_{gt}, v_t(s_i)$ Within `starting$magnitude$terms[[term]]`, `effects` contains the group or spatial effects, `variance` is $\sigma^2_{\delta,t}$ or $\sigma^2_{v,t}$, and an estimated spatial `covariance$phi` is $\phi_{v,t}$. The corresponding prior term uses `variance` for inverse-gamma shape and scale and `covariance$phi` for uniform bounds.

Composition loadings and scores Within a composition block's `terms[[term]]` entry, `loadings` contains γ for iid terms or λ for spatial terms; `scores` contains u or w, respectively. An estimated spatial `covariance$phi` has one value $\phi_{w,t,k}$ per factor column. For the term, `loading_variance` specifies $c_{\gamma,t}$ or $c_{\lambda,t}$; `covariance$phi` gives uniform bounds.

Negative-binomial size r_M The magnitude prior entry `size` gives Gamma shape and rate when size is estimated. The corresponding starting entry gives its initial value. By default size is estimated with Gamma(shape = 1, rate = 0.01) when there are no magnitude process terms. With those terms, or an explicit `size = NULL` prior setting, `control$nb_size` fixes its value.

Space-time effects use $v(s,t)$ and $w(s,t)$. Covariance entries use the names listed by `magCompCovModels`, for example `covariance$lambda` for temporal exponential decay.

Coefficient and loading prior variances and smoothing variances are fixed hyperparameters. Magnitude process variances are estimated parameters. For a composition coefficient vector $\boldsymbol{\theta}_q$, with $q = 0$ for intercepts and $q > 0$ for slopes, smoothing uses τ^2_q: `intercept_smoothing_variance` sets the intercept value and `slope_smoothing_variance` sets the common slope value within a block.

`magCompSmooth(intercept = , slope = )` controls composition intercept and slope smoothing. `samples` returns posterior arrays using the same descriptive branches and entry names as starting values. Fixed size and covariance values are omitted.

`magCompSettings` and `summary` include a `symbol` column using portable ASCII notation, such as `beta`, `sigma_delta^2`, and `phi_w`. Summary columns separately identify `component`, `block`, `term`, `parameter`, and `index`. The index identifies a coefficient, stick/covariate pair, stick/factor pair, or factor as appropriate. Parameter names are `coefficients`, `size`, `variance`, `loadings`, and covariance parameter names such as `phi`, `lambda`, and `nu`. Traceplot labels join these identifying columns and display the corresponding symbol.

See Also

`magCompPriors`, `samples`, `summary`

`magCompPrediction` *Results from fitted values and prediction*

Description

This help topic describes the named list with class `magCompPrediction`, containing requested `magnitude`, `proportion`, `density`, and `count` components, followed by `metadata`. Predictions made with `keep_effects = TRUE` also store a `latent` component for use by `samples`; its internal indexing is not part of the public interface. `standSummary` also returns this class, with a selected stand density, total, or proportion component. Selecting one type still returns a list: for example, `fitted(fit, type = "count")$count`.

Details

The definitions below describe `fitted()` and `predict()` results. See `standSummary` for the corresponding stand quantities and metadata.

Observed and new units each have two choices of target:

Units	Conditional means	Simulated observations
Observed	`fitted(target = "expected")`	`fitted(target = "observation")`
New	`predict(target = "expected")`	`predict(target = "observation")`

These calls also require the fitted object and, for prediction, `newdata`. Defaults are `target = "expected"` and `summary = "mean"`. With both model components, `type = NULL` selects all four quantities. Standalone models return the quantities described below.

For fixed-area stems, let M be overall stem density, p_j the probability of final category j, and a the sampling area. For each retained posterior draw, expected quantities are constructed as follows. Observation quantities additionally draw $N \sim \mathrm{NB}(aM, r_M)$ and then $\mathbf{Y} \sim \mathrm{Multinomial}(N, \mathbf{p})$.

Quantity	Expected	Observation
`magnitude`	M	N/a
`proportion`	p_j	Y_j/N
`density`	Mp_j	Y_j/a
`count`	aMp_j	Y_j

In a conditional two-category model, p_j is the product of the parent probability and its conditional child probability. Size r_M uses the fitted fixed value or its matching posterior draw when estimated.

Point-sampling basal-area models support both targets. Let M^{BA} be basal-area density and m the in-tree tally. Expected total tally is M^{BA}/BAF. The observation target draws

$$m \sim \mathrm{NB}(M^{BA}/\mathrm{BAF}, r_M^{BA}),$$

using the fitted fixed size or its matching posterior draw, then allocates m among categories when composition is fitted.

Expected magnitude and category density are M^{BA} and $M^{BA}p_j$; observation values are the sampled basal-area estimates $\mathrm{BAF}\, m$ and $\mathrm{BAF}\, Y_j$. Count is the expected or simulated in-tree tally, and proportion is p_j or Y_j/m, respectively. The fixed-area expressions above therefore apply with magnitude interpreted as basal-area density and exposure $a = 1/\mathrm{BAF}$; this exposure is not a physical plot area. BAF must be constant within each point but may vary between points. BAF and basal-area density use matching units, such as square feet per acre.

For `summary = "mean"`, each component is a matrix with one row per unit and one column per category, or one column named `"total"` for magnitude. For `summary = "samples"`, category quantities have dimensions unit by category by retained sample, plus a final chain dimension for multiple chains. Magnitude omits the category dimension. For `summary = "quantile"`, category quantities have dimensions unit by category by probability, and magnitude is a unit by probability matrix. Singleton dimensions are preserved; means and quantiles pool chains. Row and category labels identify the fitted units or prediction rows and final response cells.

Use `summary(result)` on a result retained with `summary = "samples"` to obtain a table of means, standard deviations, and quantiles by unit, quantity, and category. See `summary` for the returned columns. This uses the stored draws and does not rerun prediction.

For magnitude-only models, the default quantities are `magnitude` and `count`. Expected values are M and aM; observation values are N/a and N, with a negative-binomial draw for N. The count array retains a category dimension of length one named `"total"`. `density` and `proportion` are unavailable because no composition is fitted.

Composition-only models default to `proportion`. Expected probabilities need neither area nor a supplied total. Conditional expected counts are Np_j; replicate counts follow $\mathrm{Multinomial}(N, \mathbf{p})$. `fitted()` uses observed totals; `predict()` requires `total_count`. Conditional densities divide these counts by exposure. `magnitude` is unavailable. Expected probabilities remain defined at zero totals, but observation proportions are `NA`; counts and densities are zero. A supplied total is fixed across draws, not an estimate from a separate abundance model. Nested composition retains the parent/child product.

These standalone-component rules apply to both sampling families. For point sampling, conditional densities multiply tally counts by BAF, and observation proportions are undefined when the tally is zero.

Metadata contains:

`source`, `target`, `type`, `summary`, `family` The source function, target, requested quantities, summary, and response family.

`units`, `unit_ids`, `exposure` Measurement units by quantity, row identifiers, and the exposures used. Exposure can be `NULL` for predictions requiring none.

`components` Active model components.

`count_conditioning` Count conditioning: `"observed_total"`, `"supplied_total"`, or `"none"`. Composition-only CV identifies its assessment as conditional on observed totals.

`total_count` Conditioning totals by unit when used, otherwise `NULL`.

`n_samples`, `n_chains`, `n_draws` Retained draws per chain, number of chains, and their product.

`sample_ids`, `chain_ids`, `probs` Draw positions within the original retained chain, chain labels, and requested probabilities for quantile summaries (otherwise `NULL`). Draws are aligned across quantities; observation simulation does not add another draw dimension. With `predict(n_draws = ...)`, positions identify the selected subset.

`proportion_conditioning` `"positive_total"` for observation results, otherwise `"none"`. Observation proportions are undefined for zero totals. Their means and quantiles use positive-total draws only. Samples retain `NA` entries at zero totals, and all-zero units have `NA` proportion summaries.

`zero_total_probability`, `n_positive_draws` For each unit, the empirical fraction of zero-total draws and number of positive-total draws across all chains. Returned for every observation result, including type subsets; `NULL` for expected results.

All outputs within one result use the same draws. A zero simulated total gives zero magnitude, density, and count, and undefined (`NA`) proportions. Relations between draw-level quantities need not hold after separately taking means or quantiles. Expected-target intervals describe uncertainty in conditional means; observation-target intervals also include variation from the likelihood.

See Also

fitted, predict; the technical guide's Prediction and aggregation section in magComp-package.

---

magCompPriors *Set magnitude and composition priors*

Description

Creates prior specifications for magComp. Omitted settings retain automatic defaults. On a prepared model, magCompSettings shows resolved priors and editable starts.

Usage

```
magCompPriors(magnitude = list(), composition = list())
```

Arguments

magnitude	Named list overriding magnitude prior defaults; see Details.
composition	Named list overriding composition prior defaults, with optional named blocks and their terms; see Details.

Details

Resolved model settings, starting values, and extracted samples contain only active components. Supplied starting values or nondefault priors for a disabled component are rejected by magComp().

Magnitude prior settings are:

coefficient_variance Zero-mean normal variance for magnitude regression coefficients; default 100. It is independent of composition priors.

iid_variance, spatial_variance Default inverse-gamma shape and scale for each magnitude iid or spatial term's variance; both default to c(shape = 2, scale = 1).

size The default "auto" estimates negative-binomial size with c(shape = 1, rate = 0.01) when the magnitude formula has no iid or spatial terms; otherwise size is fixed at control$nb_size (default 1000). Supply a positive Gamma shape-rate pair to change the prior, or NULL to fix size explicitly. Size estimation is not allowed with magnitude random-effect terms. Composition factor terms do not impose this restriction.

terms List named by formula term names. Each entry can override variance (inverse-gamma shape and scale) and covariance, a named list of uniform(lower, upper) priors for covariance parameters.

Composition defaults are:

Setting	Value
intercept_variance	9
slope_variance	100
intercept_smoothing_variance	1

slope_smoothing_variance	4
loading_variance	1

Coefficient and unconstrained loading priors are zero-mean normal. Anchor-diagonal loadings are estimated under the same normal prior restricted to $(0, \infty)$, a half-normal prior. `loading_variance` is the underlying normal variance before truncation: one gives half-normal scale one; four gives scale two. The half-normal variance itself is $v(1 - 2/\pi)$, where v is `loading_variance`. Fixed upper-anchor zeros have no prior. Smoothing and normal prior variances are fixed hyperparameters, not sampled variances. Composition factor scores retain unit scale.

The default Gamma size prior has mean 100 and a central 95% interval of approximately 2.5 to 369. It is a proper, adjustable prior, not a noninformative one. Larger size means less extra count variation. Use `magCompSettings(model)` to inspect the resolved prior and estimation status. When size is estimated, `starting$magnitude$size` overrides its automatic start at `control$nb_size`.

A named `composition$blocks` list overrides these defaults for individual allocation blocks. Within each block, a named `terms` list can override `loading_variance` and the `covariance` prior list for each factor term. Within a factor term, the same `loading_variance` controls its half-normal diagonals and its unconstrained normal loadings. The diagonal Gibbs updates need no tuning parameter. Covariance priors may be shared across factors or specified in named lists by factor. Conditional block names use `"child|parent"`. Magnitude term priors override magnitude defaults; composition block settings override composition defaults, and factor-term settings override their block. magCompCovModels describes covariance families and automatic supports. The prior is uniform on each covariance parameter, not on an effective range.

For inverse-gamma priors, the density is proportional to $v^{-a-1}\exp(-b/v)$, with shape a and scale b. Magnitude variance updates are Gibbs steps and need no tuning. Variance-prior parameters must be positive and finite. Covariance bounds must be finite, increasing, and within their parameter domains. Unknown names, unused covariance priors for fixed parameters or iid terms, and incompatible model term/block names are rejected.

Value

A `magCompPriors` object to pass as `magComp(priors = )`.

See Also

`magCompSmooth`, `magCompSettings`, magCompNotation; the technical guide's Prior distributions section in magComp-package.

Examples

```
# Estimate size with a different Gamma prior (no magnitude random effects).
magCompPriors(magnitude = list(size = c(shape = 2, rate = 0.1)))
# Fix size at the value supplied through magCompControl(nb_size = ...).
magCompPriors(magnitude = list(size = NULL))

magCompPriors(
  magnitude = list(coefficient_variance = 25,
    terms = list(stand = list(variance = c(3, 1)))),
  composition = list(slope_variance = 4,
```

```
    blocks = list(dbh = list(intercept_variance = 2,
      terms = list(shape = list(loading_variance = 4)))))
)
```

`magCompSettings` *Inspect resolved model parameters and editable settings*

Description

Shows which parameters are estimated, fixed, or constrained; their resolved priors and initial values; and the mapping from term names to formula terms. Returns editable prior, starting-value, and tuning templates without running the sampler.

Usage

```
magCompSettings(object)
```

Arguments

`object`	A prepared or fitted `magComp` model.

Details

Prepare a model with `magComp()` and `control = magCompControl(run_sampler = FALSE)`. Inspect `magCompSettings(model)`, edit the returned `priors` and/or `starting` lists, and pass them to a new `magComp()` call with the same formulas and data.

The returned prior template explicitly records resolved block/term settings. Editing a global default does not override an explicit block/term entry; edit the relevant entry or remove it to inherit a default. The initial values shown for fitted objects are their original starts, not posterior summaries. Use `magCompStarting(fit, use = "last")` or `use = "mean"` to extract starts from a previous fit.

Constraints on factor scales and loadings, fixed smoothing variances, and the magnitude-size estimation restriction are displayed. Priors and starts can be overridden selectively. Unsupported names and dimensions are rejected.

Value

A `magCompSettings` object containing `parameters` (a data frame of parameter settings, including a manuscript `symbol` column in ASCII), `terms` (formula term names and labels), `priors` (a resolved `magCompPriors` object), and `starting` (a complete initial-value template), plus `tuning` (initial covariance proposal standard deviations). Printing shows the two tables.

See Also

magCompNotation for mathematical symbols and software names.

Examples

```
rows <- expand.grid(plot = paste0("p", 1:8), cls = c("a", "b"))
rows$count <- rep(1:8, 2)
rows$area <- 1
rows$stand <- rep(rep(c("s1", "s2"), each = 4), 2)
dat <- magCompData(rows, unit = plot, category = cls,
                   tree_count = count, design = fixedArea(area))
model <- magComp(dat, magnitude = ~ iid(stand, name = "stand"),
                 control = magCompControl(run_sampler = FALSE))
settings <- magCompSettings(model)
settings
settings$priors$magnitude$terms$stand$variance <- c(shape = 3, scale = 1)
settings$starting$magnitude$terms$stand$variance <- 0.5
fit <- magComp(dat, magnitude = ~ iid(stand, name = "stand"),
               priors = settings$priors, starting = settings$starting,
               control = magCompControl(n_samples = 10, n_burn = 5,
                 chain_seeds = 7, verbose = FALSE))
magCompStarting(fit, use = "last")
```

`magCompSmooth` *Smooth composition coefficients across ordered categories*

Description

Specifies how regression coefficients vary across ordered composition categories. Smoothing requires ordered categories in `magCompData`. Unspecified methods default to independent coefficient priors.

Usage

```
magCompSmooth(intercept = c("none", "rw1", "rw2"),
              slope = c("none", "rw1", "rw2"),
              loading = c("none", "rw1", "rw2"),
              ordered = TRUE, ...)
```

Arguments

`intercept`	Smoothing for composition intercept coefficients: independent (`"none"`), first-order (`"rw1"`), or second-order (`"rw2"`) random-walk smoothing.
`slope`	Smoothing for each composition slope, with the same choices.
`loading`	Loading smoothing. Only `"none"` is implemented.
`ordered`	Ordering flag stored in this specification. When preparing a model, the category ordering recorded by `magCompData()` determines whether smoothing is permitted. This flag does not order the data.
`...`	Named block or response overrides. Each is a list containing `intercept`, `slope`, `loading`, and/or `ordered`, or a single method name applied to both intercept and slope. Conditional block names have the form `"child|parent"`.

Details

RW1 penalizes adjacent coefficient differences; RW2 penalizes changes in those differences. In conditional blocks, smoothing acts separately within each parent's child-category sequence. It does not smooth across parents.

Global settings apply unless overridden by a named block or response entry. An exact block name takes precedence over its response name. Smoothing variances are fixed prior settings controlled by `magCompPriors`; they are not sampled parameters. The technical guide's section on regression coefficients and ordered-category smoothing gives the precision matrices; see magComp-package.

Value

A `magCompSmooth` object to pass as `magComp(smooth = )`.

See Also

`magCompComposition`, `magCompPriors`

Examples

```
magCompSmooth(intercept = "rw2", slope = "rw2")
magCompSmooth(dbh_class = list(intercept = "rw2", slope = "rw1"))
```

`magCompStarting` *Inspect initial values or extract starts from posterior samples*

Description

Returns a starting-value list organized by component, block, and term, as in `magCompSettings`. Pass it to `magComp(starting = )`. Extracting posterior values initializes a new run; it does not resume RNG state or Metropolis adaptation from an earlier run.

Usage

```
magCompStarting(object, ...)

## S3 method for class 'magComp'
magCompStarting(object,
  use = c("initial", "last", "mean"), chain = 1, ...)
```

Arguments

`object`	A prepared or fitted `magComp` model.
`use`	`"initial"` returns the resolved values used to initialize the model. `"last"` extracts the selected chain's last retained draw; `"mean"` extracts posterior means across all retained chains. The latter two choices require a fitted model.
`chain`	Chain index for `use = "last"`.
`...`	Currently unused.

Details

Resolved model settings, starting values, and extracted samples contain only active components. Supplied starting values or nondefault priors for a disabled component are rejected by `magComp()`.

Estimated covariance-parameter starts must lie strictly between their uniform-prior bounds because their Metropolis proposals use a bounded-logit transformation. Starting values have the following structure:

```
list(
  magnitude = list(
    coefficients = named_vector,
    size = positive_scalar,  # only when size is estimated
    terms = list(term_name = list(
      effects = named_vector, variance = positive_scalar,
      covariance = list(phi = value)  # spatial/space-time terms
    ))
  ),
  composition = list(blocks = list(block_name = list(
    coefficients = named_matrix,
    terms = list(term_name = list(
      loadings = named_matrix, scores = named_matrix,
      covariance = list(phi = value)  # one value per factor
    ))
  )))
)
```

Only the entries a user wants to override need to be supplied. Omitted entries receive automatic starts. Obtain the correct term names, dimensions, and dimnames from a prepared model via `magCompSettings(model)$starting`. Use `name = "stand"` inside a formula term to give it a stable name.

Coefficient matrices have one row per coefficient stick and one column per fixed-effect design column. Loading matrices have coefficient-stick rows and factor columns; score matrices have group/location rows and factor columns. Named inputs are matched to model names, including reordered vectors and matrix rows/columns. Unnamed inputs must follow the template's order. Inputs for coefficients, effects, scores, and loadings must have exactly the required dimensions and finite values; scalar recycling is not performed for these arrays. Loading starts must have positive values on the anchor diagonal and zeros above it. Supplied valid loading and score starts are preserved.

Automatic factor starts use residuals from regressions of transformed empirical continuation ratios. Conditional child blocks have separate regressions for each parent. Terms are initialized sequentially, with iid terms before spatial terms and formula order within each group. Each term's contribution is removed before initializing the next term by SVD.

Automatic loadings and scores are rotated and scaled to satisfy the anchor constraints and give score columns unit mean square. This is an initialization convention, not a constraint on sampled scores. Valid user-supplied loadings or scores are preserved; an omitted matrix is initialized automatically. The technical guide's Data preparation, aggregation, and initialization section gives the details; see magComp-package.

Magnitude iid/spatial variance starts use the variance of initialized effects, bounded below by `1e-4`, with a fallback of one if nonpositive or undefined. Covariance starts have automatic coordinate-based defaults. Estimated values must lie strictly inside their uniform-prior supports. To fix a covariance parameter, wrap its starting value in `fixed()`, for example `covariance = list(phi = fixed(2))`. Fixed markers are preserved when extracting starts from a fitted model. Common scalars apply to all factors; named vectors or lists specify individual factors. See magCompCovModels.

Value

A named list with `magnitude` and `composition` branches. Initial templates contain all fitted model parameter blocks, including fixed covariance values. Extracted starts require the same model structure and group/location labels in the next fit.

See Also

magCompNotation for the symbols corresponding to each entry; `samples` for all retained draws in matching branches.

`magCompTerms` *Formula terms for magnitude and composition models*

Description

Formula-only markers for magnitude iid, GP, and NNGP effects and composition iid, GP, and NNGP factor terms. These functions are parsed inside model formulas.

Usage

```
iid(...)
gp(...)
nngp(...)
factor_iid(...)
factor_gp(...)
factor_nngp(...)
```

Arguments

`...`	Grouping or coordinate column names and term settings described below.

Details

Use `iid(group)` for magnitude group effects, `gp(x, y)` for a dense Gaussian process, or `nngp(x, y, m = 15)` for a nearest-neighbor process. Each magnitude process has its own estimated variance. Composition uses `factor_iid(group, K = 1)`, `factor_gp(x, y, K = 1)`, or `factor_nngp(x, y, K = 1, m = 15)`. Its factors have unit variance. Unconstrained loadings have normal priors; positive anchor-diagonal loadings have half-normal priors.

For each spatial term, different sampling units must have distinct coordinate vectors. Exact duplicate locations cause an error during model preparation that identifies the term and affected unit IDs. Nearby but distinct locations are allowed. Multiple tree records within one unit and the repeated unit rows in conditional allocation blocks are allowed; those rows share the unit's spatial effects. This restriction applies to fitting. Repeated prediction coordinates share a spatial-effect draw as described in `predict`.

Spatial calculations use the specified correlation matrices without diagonal additions or conditional-variance clipping. Failed covariance calculations report an error during initialization, sampling, or prediction.

All terms accept `name`, a unique nonempty name within the component or allocation block. Otherwise names such as `iid_1` and `gp_1` follow formula order within each type. Use these names for overrides in `magCompPriors` and `starting`; inspect the resolved names with `magCompSettings`.

Spatial terms accept `cov_model`: `"exp"` (default), `"matern"`, `"sep_exp"`, `"multi_res_sep_exp"`, or `"gneiting"`. For a space-time family, supply numeric time as the final coordinate, for example `nngp(x, y, year, cov_model = "sep_exp", m = 15)`. Repeated sites at *different* times are then allowed; repeated full space-time coordinates for different sampling units remain an error. A spatial-only term on revisited sites is not supported.

NNGP neighbors use Euclidean spatial distance and absolute time separation, combined as $\sqrt{h^2 + (su)^2}$, where `st_scale = s` defaults to one. This positive multiplier has spatial units per time unit and affects only neighbor selection. Covariance calculations use the original spatial and temporal lags. Ordering and neighbors remain fixed during fitting. Prediction inherits the same setting and conditions jointly on observed and earlier new locations.

Covariance priors, starts, and tuning use each term's `covariance` list. For example, `covariance = list(phi = uniform(0.1, 5))` sets a prior, `covariance = list(phi = 1)` sets a start, and `covariance = list(phi = fixed(1))` fixes that parameter. See magCompCovModels for the kernels, parameter names, and defaults.

Factor terms accept `anchors` (coefficient-stick names or row indices) and `factor_names`. Anchor rows define zeros above the anchor diagonal and positive, estimated diagonal entries. Automatic anchors use variation in observed continuation ratios. Default factor names are the anchor categories. Factor rank and anchor constraints are model specifications; diagonal loadings are estimated under half-normal priors controlled by `loading_variance`. The sampler updates unrestricted working loadings, then jointly changes the sign of each loading column and its factor scores when needed to make the anchor positive. This preserves the factor contributions and the specified priors; returned draws satisfy the anchor constraints.

Colon interactions such as `height:iid(stand)` or `height:factor_nngp(x, y, K = 1)` multiply the latent contribution by the covariate. Include `height` separately for an average regression slope. Multiple terms may be used in each component or allocation block.

Named arguments are limited to those described above. Unknown arguments are rejected. Set priors and initial values through their dedicated interfaces.

Value

These functions stop if evaluated outside a model formula. They specify terms for `magComp()` to interpret; they do not generate effects themselves.

See Also

magCompPriors, magCompStarting, magCompNotation; the technical guide's Model terms and Factor identification sections in magComp-package.

Examples

```
# These are formula objects, to be supplied to magComp().
~ height + iid(stand, name = "stand") + nngp(x, y, m = 15)
~ height + factor_nngp(x, y, K = 2, name = "spatial")
```

---

plot.magComp *Traceplots for magComp fits*

Description

`plot(fit)` draws base-R traceplots for monitored scalar parameters in a fitted `magComp` object. PDF output can contain multiple pages; PNG output is limited to the first page. Titles include the manuscript symbol alongside the descriptive identifier. The horizontal axis uses original MCMC iteration numbers. Pooled sample objects are separated into their original chains for plotting. `plot(fit, ...)` delegates to `plot(samples(fit, ...))`; use `type = "effects"` or `type = "combined"` to examine those quantities.

Usage

```
## S3 method for class 'magComp'
plot(x, parameters = NULL, file = NULL,
                      n_col = 2, n_row = 3,
                      width = 8, height = 8, res = 150, ...)

## S3 method for class 'magCompSamples'
plot(x, parameters = NULL, file = NULL,
                             n_col = 2, n_row = 3,
                             width = 8, height = 8, res = 150, ...)
```

Arguments

`x`	A fitted `magComp` object or extracted `magCompSamples` object.
`parameters`	Character vector selecting parameter names, indices, or full identifiers. Examples are `"variance"`, `"(Intercept)"`, and `"magnitude:stand:variance"`. Full identifiers join nonempty component, block, term, parameter, and index fields with colons.
`file`	Optional output file ending in `.pdf` or `.png`.
`n_col`	Number of plot columns per page.
`n_row`	Number of plot rows per page.
`width`	Graphics device width in inches.

height	Graphics device height in inches.
res	PNG resolution.
...	For a fitted model, arguments passed to samples(). Unused for a sample object.

Value

Invisibly returns the output file path when file is supplied, otherwise the plotted parameter labels.

See Also

summary for the monitored parameters; samples for selection and combined coefficients.

---

ppcSummary *Posterior predictive check summaries*

Description

Compares observed counts with posterior predictive count samples from a fitted magComp model. Replicates are obtained from

```
fitted(object, target = "observation", type = "count",
       summary = "samples")$count
```

These use the original exposures when magnitude is fitted. fitted() returns these simulated observations; ppcSummary() compares replicate count summaries with the observed counts. Each call generates fresh replicates; it does not reuse a previously created prediction object or refit the model.

Usage

```
ppcSummary(object, ...)

## S3 method for class 'magComp'
ppcSummary(object,
                              type = NULL,
                              group = NULL,
                              probs = c(0.025, 0.5, 0.975), ...)
```

Arguments

object	A fitted magComp object.
type	Summary scale: total count by sampling unit, total count by category, sampling-unit/category cell count, total count by group, or group/category count. The default is NULL: use "unit_total" when magnitude is fitted. Composition alone uses "unit_category".
group	Grouping variable required for "group_total" or "group_category". This can be a column name in the unit-level data or a vector with one value per sampling unit.
probs	Posterior predictive quantile probabilities.
...	Must be empty. Unknown arguments are rejected.

Details

Choose one supported `type`. Full models support all five checks. Magnitude-only models support `"unit_total"` and `"group_total"`, using negative-binomial replicate totals. Composition-only models support `"unit_category"`, `"category_total"`, and `"group_category"`, using multinomial allocations conditional on observed totals. Checking those fixed totals would not assess fit, so total checks are rejected. The printed output states this conditioning. The `count_conditioning` attribute also records it. Both fixed-area stems and point-sampling basal-area models are supported. For point sampling, checks compare observed and replicated in-tree tallies using each point's BAF. Group summaries add the observed counts and replicate counts from units in each group. These remain tally-count checks, including when BAF varies between points; they are not expanded stand estimates.

For each check, T is the observed count or sum of counts selected by `type`; T^{rep} is its replicate counterpart. The reported `p_lower` and `p_upper` estimate

$$p_{lower} = \Pr(T^{rep} \leq T \mid \text{data}), \qquad p_{upper} = \Pr(T^{rep} \geq T \mid \text{data}).$$

They are computed as fractions of replicates across all retained draws and chains. The `p_value` column is a two-sided posterior predictive p-value:

$$p_{two} = \min\{1,\, 2\min(p_{lower}, p_{upper})\}.$$

Ties contribute to both tails, so their probabilities need not sum to one.

A small `p_lower` indicates an unusually low observed count relative to the replicates; a small `p_upper` indicates an unusually high count. For example, `p_upper = 0.02` gives `p_value = 0.04` when the upper tail is the smaller one. A `p_value` near one indicates a central observed count. A reported zero means no simulated replicates reached that tail; it is not evidence that its probability is exactly zero.

These are Bayesian posterior predictive checks, not probabilities that the model is correct or incorrect. The same data inform the fit and the check; posterior predictive p-values are not generally uniform under the fitted model (Gelman, 2013), and a threshold such as 0.05 does not have the usual hypothesis-test calibration. Use small values to flag discrepancies for inspection, including their size and direction. Large values do not establish overall model adequacy. These checks concern the fitting data, not cross-validation performance.

The requested quantiles describe replicate observations, including sampling variation. They are not credible intervals for expected counts. Separately summarizing draws returned by `fitted()` gives the same kinds of quantities, with Monte Carlo differences because separate calls simulate separate replicates.

Replicate simulation uses R's current random-number state. Use `set.seed()` externally for reproducibility.

Value

A data frame with identification columns `type`, `group`, `unit`, and `category`, plus:

observed The observed count or sum of counts.

mean The posterior predictive replicate mean.

q... Requested replicate quantiles; defaults are `q2_5`, `q50`, and `q97_5`.

p_lower, p_upper Fractions of replicates at or below, and at or above, the observed value.

p_value Twice the smaller tail fraction, capped at one.

n_draws The number of replicate draws pooled across chains.

References

Gelman, A. (2013). Two simple examples for understanding posterior p-values whose distributions are far from uniform. *Electronic Journal of Statistics*, 7, 2595–2602. doi:10.1214/13EJS854.

See Also

`fitted`, `magCompCV`

`predict.magComp` *Expected values or observations at new sampling units*

Description

Constructs posterior predictions at new units, including every fitted regression, group, spatial, and composition-factor term. See magCompPrediction for the returned quantities and the distinction between expected values and simulated observations.

Usage

```
## S3 method for class 'magComp'
predict(object, newdata,
                          type = NULL,
                          target = c("expected", "observation"),
                          summary = c("mean", "samples", "quantile"),
                          area = NULL, probs = c(0.025, 0.5, 0.975),
                          total_count = NULL, n_draws = NULL,
                          keep_effects = FALSE, ...)
```

Arguments

`object`	A fitted `magComp` object.
`newdata`	A data frame with one row per prediction unit, containing the covariates, group labels, and coordinates required by the fitted model.
`type`	Quantities to return; see magCompPrediction for supported choices. With `NULL`, defaults depend on the fitted components.
`target`	Use `"expected"` for conditional means based on posterior and new latent-effect draws, or `"observation"` to simulate observations from the fitted likelihood.
`summary`	Return means, aligned retained samples, or quantiles. Means and quantiles pool all retained draws across chains.

`area` Prediction exposure: a positive finite scalar, a numeric vector with one value per row, or the name of a column in `newdata`. For `fixedAreaStems`, exposure is the sampling area for each predicted count. If omitted, the area column named in the fitted design must be in `newdata`. For `pointBasalArea`, supply exposure `1 / BAF`, or omit `area` and supply the fitted design's BAF column in `newdata`. Training exposures are never silently reused. Exposure is required for count predictions or observation simulation when magnitude is fitted. For composition-only models, it is required only for conditional density; probabilities and conditional counts need no exposure.

`probs` One or more probabilities strictly between zero and one, used for `summary = "quantile"`.

`total_count` For composition-only models, a numeric vector with one nonnegative integer total per prediction row, or the name of its column in `newdata`. Named vectors are matched to prediction unit IDs; unnamed vectors follow row order. Counts must not exceed R's integer limit. Required for conditional counts, conditional densities, or any observation target. Totals are treated as known and are not simulated. Not used when magnitude is fitted.

`n_draws` Number of retained posterior draws to use per chain. The default `NULL` uses all retained draws. A positive integer selects approximately evenly spaced draws from each chain and must not exceed the number available. Selection preserves matching parameters and latent effects, and does not change the fitted object.

`keep_effects` Retain the parameter and latent-effect draws used for prediction, for extraction with `samples`. Requires `summary = "samples"`. Defaults to `FALSE` to avoid storing additional arrays. Both expected and observation targets are supported.

`...` Must be empty. Unknown arguments are rejected.

Details

Prediction draws new latent effects conditional on each retained posterior draw. Thus even `target = "expected"` can require random simulation. With both components, `target = "observation"` additionally draws a negative-binomial total and multinomial allocation. Magnitude-only models draw only the total. Composition-only models instead allocate `total_count` using a multinomial draw. Their expected category counts are $N_0 p_{0j}$, conditional on the supplied total N_0; no magnitude is estimated. Expected probabilities require neither totals nor areas. Use `set.seed()` externally for reproducibility.

Use `n_draws` to reduce prediction time and memory while keeping all fitting draws for parameter summaries and diagnostics. For example, `n_draws = 100` with three fitted chains generates 300 predictive draws. Selection occurs before prediction and retains every requested location in each draw, including joint spatial effects. Selection itself uses no random numbers. Output `metadata$sample_ids` identifies the positions selected within the original retained chains. Fewer draws increase Monte Carlo variability, especially in interval endpoints.

With `keep_effects = TRUE`, `samples(predictions, type = "effects")` extracts the stored new-unit effects, and `type = "combined"` adds their matching regression coefficients and loading-times-score contributions. These are the same draws used to construct the response predictions; extraction does not simulate again. All fitted latent terms are generated and retained, even when `type` requests only magnitude. `standSummary()` aggregates response draws and does not carry these location-level effects into its output.

Observed iid groups reuse posterior effect draws. Each unseen group receives one new Gaussian effect per term and posterior draw, shared by all rows in that group. Magnitude terms use their sampled variances; composition factor scores use unit variance. For space-time terms, a location includes the final time coordinate: an exact location-time match reuses its fitted effect, while a revisit at a new time receives a conditional draw. Prediction inherits the fitted covariance family and `st_scale` for neighbor searches. Known spatial locations reuse fitted effects, and repeated new coordinates share one sampled spatial effect per term and draw. Dense-GP prediction draws new effects jointly from their conditional Gaussian distribution. NNGP prediction orders new locations by their first coordinate, using subsequent coordinates to break ties, and uses neighbors among observed and earlier new locations in the same call. There is no conditionally independent spatial option. The NNGP approximation depends on the requested location set; separate calls omit cross-call links and do not reproduce a single joint call. Supply the complete desired support in one call when its joint uncertainty matters.

Both targets support `fixedAreaStems` and `pointBasalArea`. For point sampling, the expected total tally is M_0^{BA}/BAF_0; multiplying simulated tallies by BAF gives sampled basal-area estimates. Composition-only models condition on the supplied tally total. For an observed location, prediction reuses its stored spatial effect, but covariates and exposure still come from `newdata` and `area`. Use `fitted()` to obtain results for all original units with their original inputs.

`standSummary` aggregates stored density or magnitude samples using explicitly supplied represented land areas. It preserves the selected target. These weights serve a different purpose from the sampling exposures used to generate counts here. Neither function determines areas from polygons.

Value

A `magCompPrediction` named list with requested quantity components and `metadata`, even for one requested quantity. All quantities share posterior draw and chain identities. See that help page for dimensions and metadata fields.

See Also

`summary` for summarizing stored draws; `fitted`, `standSummary`, `magCompCV`

`samples` *Extract posterior draws and combined coefficients*

Description

Extracts retained posterior draws without running the sampler or generating predictions. The result has `summary()` and `plot()` methods. Draw selection is synchronized across every parameter and latent effect.

Usage

```
samples(object, ...)

## S3 method for class 'magComp'
samples(object, type = c("parameters", "effects", "combined"),
```

```
                              start = 1, thin = 1, chains = NULL,
                              combine_chains = FALSE, group = NULL, ...)

## S3 method for class 'magCompPrediction'
samples(object,
                  type = c("parameters", "effects", "combined"),
                  start = 1, thin = 1, chains = NULL,
                  combine_chains = FALSE, group = NULL, ...)
```

Arguments

object	A fitted magComp model, or the result of predict(..., summary = "samples", keep_effects = TRUE).
type	"parameters" returns all raw parameter arrays in the component/block/term structure below. Its default summary reports scalar parameters. "effects" selects individual magnitude effects and composition factor scores, including them in summaries and traceplots. "combined" constructs fixed coefficients plus their matching random coefficients at fitted or predicted units. Only coefficients with random-effect terms are returned.
start	First saved draw to retain within each chain, counted after the burn-in and thinning already applied by the fit. Defaults to 1.
thin	Additional thinning interval among saved draws. Defaults to 1.
chains	Chain numbers or names to select, in the requested order. NULL selects all chains.
combine_chains	Pool selected chains after constructing quantities. Original draw and chain identities are retained for summaries and diagnostics.
group	For type = "combined", a column name in the fitted unit data or prediction newdata for averaging within groups, or ".all" for averaging over all units. NULL returns each unit. Averages give each unit equal weight and are calculated within each draw.
...	Additional arguments, currently unused.

Details

For prediction objects, the stored latent draws are those used to construct the response predictions, before any negative-binomial or multinomial draws. Extraction does not generate new effects. The parameter arrays contain the matching subset selected by predict(n_draws = ...); draw identities and original MCMC iterations are preserved. start and thin select positions within that subset. Magnitude effects are indexed by new unit; spatial scores are indexed by new location, and iid composition scores by allocation-block row (including parent context for conditional blocks). Response draws themselves remain available in the prediction object's magnitude, proportion, density, or count components.

Resolved model settings, starting values, and extracted samples contain only active components. Supplied starting values or nondefault priors for a disabled component are rejected by magComp().

For type = "parameters", the returned structure is:

```
list(
```

```
  magnitude = list(
    coefficients = coefficient_draws,
    size = size_draws,                  # only when estimated
    terms = list(term_name = list(
      effects = effect_draws, variance = variance_draws,
      covariance = list(phi = phi_draws) # estimated parameters only
    ))
  ),
  composition = list(blocks = list(block_name = list(
    coefficients = coefficient_draws,
    terms = list(term_name = list(
      loadings = loading_draws, scores = score_draws,
      covariance = list(phi = phi_draws) # estimated parameters only
    ))
  )))
)
```

Magnitude coefficients and effects are parameter-by-sample matrices. Scalar variance and size arrays have one parameter row; each covariance parameter has an array with one row per factor for which it is estimated. Composition coefficient arrays are stick-by-design-column-by-sample. Loading arrays are stick-by-factor-by-sample; score arrays are group-or-location-by-factor-by-sample. Multiple chains add a final chain dimension to every array. Parameter dimensions and singleton dimensions are retained, and dimnames identify coefficients, sticks, groups, locations, factors, and chains. Sample positions align across the result.

Fixed size and covariance values are omitted from samples. Fixed covariance values are retained by `magCompStarting()` for restarting the same model. Use `magCompSettings` to inspect these fixed values. Loading arrays retain the fixed upper-anchor zeros and estimated positive diagonals. Factor variances and smoothing variances are fixed controls and have no posterior sample arrays.

For example, `draws$magnitude$terms$stand$variance` contains the posterior draws of the variance whose prior is specified by `priors$magnitude$terms$stand$variance` and whose initial value is `starting$magnitude$terms$stand$variance`. The prior entry holds inverse-gamma shape and scale; the starting entry holds a single variance.

Use `fitted` and `predict` for draws of response quantities. Use `magCompStarting` with `use = "last"` or `use = "mean"` to turn parameter samples into a starting-value list.

Value

An object of class `magCompSamples`. Raw parameters and effects are nested lists of arrays. Combined coefficients are a named list of saved-draw-by-chain matrices, or vectors when chains are pooled. Their `info` attribute identifies component, block, coefficient, allocation row, parent context, and fitted or predicted unit or group.

The `sample_info` attribute records type, retained draw positions, original MCMC iteration numbers, chain identities, grouping, and pooling. Pooled draws are concatenated one chain at a time in the selected order; every chain uses the same selected draw positions.

Combined coefficients

For a magnitude random intercept with both group and spatial terms, the combined coefficient is $\beta_0 + \delta_{g(i)} + v(s_i)$. A varying slope instead combines its fixed slope with the random coefficients attached to that covariate. A missing fixed counterpart contributes zero.

Composition combinations sum the loading-times-score contributions across all factors and matching terms, then add the corresponding fixed coefficient. They support iid, GP, and NNGP terms and retain conditional parent contexts. Loadings and scores are multiplied within matching draws and chains. Covariates are not multiplied into these coefficients. Response means and observation draws belong to `fitted()` and `predict()`.

Group averages summarize coefficients over the supplied units, not forest density or abundance. Use `standSummary()` for area-weighted response summaries. Where several latent terms contribute to one coefficient, all matching terms are included. These are combined coefficients, not separately attributed term effects. New groups or locations require `predict()` with `keep_effects = TRUE`, followed by `samples()` on its result.

See Also

magCompNotation, `magCompSettings`, `summary`

Examples

```
rows <- expand.grid(plot = paste0("p", 1:4), cls = c("a", "b"))
rows$count <- rep(1:4, 2)
rows$area <- 1
rows$stand <- rep(c("s1", "s1", "s2", "s2"), 2)
dat <- magCompData(rows, unit = plot, category = cls, tree_count = count,
                   design = fixedArea(area))
fit <- magComp(dat, magnitude = ~ iid(stand, name = "stand"),
  control = magCompControl(n_samples = 6, n_burn = 3,
    verbose = FALSE))
draws <- samples(fit)
draws$magnitude$coefficients
draws$magnitude$terms$stand$variance
summary(fit, type = "effects")
combined <- samples(fit, type = "combined", group = "stand")
summary(combined)
# Equivalent convenience call:
summary(fit, type = "combined", group = "stand")
```

`spatialDiagnostics` *Spatial diagnostics for magComp fits*

Description

Summarizes monitored spatial scalar parameters and simple posterior dependence diagnostics from a fitted `magComp` object. The diagnostics are intended to flag weak range identification and confounding between spatial range parameters, process variances, and composition factor loading magnitudes.

Usage

```
spatialDiagnostics(object, ...)

## S3 method for class 'magComp'
spatialDiagnostics(object, probs = c(0.025, 0.5, 0.975), ...)
```

Arguments

object	A fitted magComp object.
probs	Posterior quantile probabilities.
...	Currently unused.

Details

For each spatial or space-time term, the summary reports covariance parameters under their registry names. Magnitude terms also report process variance; composition factors report loading-column norms. Acceptance rates refer to one joint covariance block per magnitude process or composition factor. Posterior correlations compare each estimated covariance parameter with the magnitude variance or composition loading norm. Fixed values are summarized without acceptance or correlation rows for an entirely fixed block.

Value

A list of data frames with elements summary, acceptance, and correlations. The object has class magCompSpatialDiagnostics and a compact print method.

See Also

magCompNotation for the full parameter mapping.

---

standSummary *Summarize predictive densities for stands, strata, or forests*

Description

Aggregates existing density draws using represented land areas. No new predictions are generated: the supplied expected or observation target and posterior draw alignment are preserved.

Usage

```
standSummary(object, ...)

## S3 method for class 'magCompPrediction'
standSummary(object, data, unit, group = NULL, represented_area,
                            type = c("density", "total", "proportion"),
                            summary = c("mean", "samples", "quantile"),
                            probs = c(0.025, 0.5, 0.975), ...)
```

Arguments

object	A magCompPrediction result from fitted() or predict(), retained with summary = "samples" and containing density or magnitude. Both sampling families are supported.
data	A data frame or tibble with one row for each prediction location's contribution to a stand or other group. A cell crossing stand boundaries has a separate row and represented area for each stand. Rows may be in any order and may select a subset of prediction locations.
unit	The column in data containing prediction location IDs, supplied as a bare name or quoted column name. IDs must match the units stored in object; the column name can differ from the original ID column.
group	The column in data identifying stands or other groups, supplied as a bare name or quoted column name. Omission or NULL aggregates all rows into one group named "all". Groups appear in order of first occurrence.
represented_area	The column in data containing positive finite land areas, supplied as a bare name or quoted column name. For cells crossing stand boundaries, use the cell's area within that row's stand. Use units consistent with the density, such as hectares for trees per hectare.
type	One stand quantity: area-weighted density (the default), total, or category proportion. Proportions require category density samples.
summary	Return posterior means, aligned retained samples, or quantiles. Means and quantiles pool retained draws across chains.
probs	Probabilities strictly between zero and one, used for summary = "quantile".
...	Must be empty. Unknown arguments are rejected.

Details

For each category and draw, total is the sum of density times represented area. Density is that total divided by the group's summed area. Proportions divide category totals by their sum across categories. They describe pooled stand composition, not an area-weighted average of local proportions.

Full models supply category density draws. Magnitude-only models supply magnitude, giving overall density or total under a single category named "total". When both quantities are present, category density is used. Composition-only models can supply conditional category densities: fitted() uses observed totals and predict() requires total_count, with plot area or BAF supplied for the density conversion. Those totals remain fixed across draws, and the conditioning is recorded in metadata. Proportions alone lack the abundance information needed for pooled stand composition and cannot be aggregated by this function.

For fixed-area sampling, stand densities are stems per unit area and totals are stems. For point sampling, densities are basal area per unit area and totals are basal area. Point-sampling fitted() and predict() already multiply category tallies by BAF; standSummary() does not apply BAF again. Point-sampling proportions describe basal-area composition.

Represented areas within a group define its summary area. For a cell grid, intersect the cells with the target polygons and supply the intersection areas. Each cell's same density draw contributes to every group it overlaps, treating that density as constant within the cell. Multiple rows for the same

location and group add their represented areas, as for disconnected intersection pieces; include each piece once. Omitting `group` pools all supplied areas. For grid points, equal weights can be a stand's polygon area divided by its number of points. These are aggregation weights, distinct from plot areas or reciprocal BAF used to generate counts. No areas are inferred from polygons, grid spacing, or sampling exposure. Fitted locations also require explicit represented areas; sampled plots do not automatically represent a whole stand.

The same calculation applies to predictions from regression, stratum and spatial models. Their effects are already included in the supplied draws. Supply all locations for the desired support in one prediction call when joint spatial uncertainty matters. Aggregation preserves dependence already in those draws but cannot reconstruct dependence omitted by separate calls. When model relationships differ by stratum, prediction units must represent the appropriate stratum; aggregation weights do not change the fitted effects.

Internally, a sparse group-by-location area matrix multiplies the density samples, preserving category, draw and chain alignment without constructing a long table of draws. Density divides each group's totals by its summed area.

Totals and densities retain zero-valued draws. Proportions are `NA` for zero-total draws; their means and quantiles condition on a positive total. If no positive draws remain, proportion summaries are `NA`. Metadata records the zero-total fraction and number of positive draws by stand.

Value

A `magCompPrediction` named list containing the selected `density`, `total`, or `proportion` component and `metadata`. Means are stand-by-category matrices, samples are stand-by-category-by-draw arrays with the input chain dimension retained, and quantiles are stand-by-category-by-probability arrays. Magnitude inputs use a category named `"total"`. Stand and category labels identify the rows and columns. Use `summary(result)` on retained stand samples for a table of means, standard deviations, quantiles, and contributing draw counts.

Metadata records the original target, family, units, count conditioning, draw and chain identities, stand IDs in `unit_ids`, and summed `represented_area` by stand. `prediction_source` identifies `fitted` or `predict`. The `aggregation` list retains the original prediction IDs, sampling exposures, and any conditioning totals. Its `data` element records the supplied mapping with columns `unit`, `group`, and `represented_area`; these are location contributions, not stand totals.

See Also

`fitted`, `predict`, `summary`; the technical guide's Prediction and aggregation section in magComp-package.

Examples

```
## Not run:
# Plot areas determine counts; intersection areas weight densities.
pred <- predict(fit, grid, area = "plot_area", target = "observation",
                type = "density", summary = "samples")
stands <- standSummary(pred, data = cell_stand_areas,
                       unit = prediction_id, group = stand_id,
                       represented_area = area_ha, summary = "samples")
summary(stands)
stands$density
```

```
# One row per cell is sufficient for a forest summary.
standSummary(pred, data = grid, unit = prediction_id,
             represented_area = cell_area_ha, summary = "quantile")

# The same interface handles basal-area densities from point sampling.
point_pred <- predict(point_fit, grid, target = "observation",
                      type = "density", summary = "samples")
standSummary(point_pred, data = grid, unit = prediction_id,
             group = stand_id, represented_area = expansion_area_acres,
             type = "total")$total


## End(Not run)
```

`summary.magComp` *Posterior parameter summaries*

Description

Summarizes posterior draws. `summary(fit, ...)` calls `samples(fit, ...)` and then summarizes that sample object. The default reports scalar parameters; use `type = "effects"` or `type = "combined"` for individual effects or combined coefficients. Multiple-chain fits include the classical unsplit Gelman-Rubin $\widehat{R}$ diagnostic; single-chain fits report `NA` for $\widehat{R}$. Effective sample size is computed from within-chain autocorrelation and summed across chains, without a between-chain correction. The R-hat calculation is bounded below by one and does not split or rank-normalize chains. See the technical guide's Diagnostics section in magComp-package for the formulas.

Usage

```
## S3 method for class 'magComp'
summary(object, probs = c(0.025, 0.5, 0.975), ...)

## S3 method for class 'magCompSamples'
summary(object, probs = c(0.025, 0.5, 0.975), ...)
```

Arguments

`object`	A fitted `magComp` object or an object returned by `samples()`.
`probs`	Nonempty numeric vector of posterior quantile probabilities strictly between zero and one.
`...`	For a fitted model, arguments passed to `samples()`, including `type`, `start`, `thin`, `chains`, `combine_chains`, and `group`. Unused for an already extracted sample object.

Details

Parameter names match the public sample entries: `coefficients`, `size`, `variance`, `loadings`, and covariance parameter names such as `phi`, `lambda`, and `nu`. The `index` column identifies the relevant design column, stick/covariate pair, stick/factor pair, or factor. Component, block, and term identities stay the same as in `magCompSettings` and `samples`. Symbols use portable ASCII notation. Fixed size and covariance values are excluded; positive anchor-diagonal loadings are included as estimated parameters. Fixed upper-anchor zeros are omitted from the scalar summary. Latent effects and scores are available through `summary(fit, type = "effects")`. Combined coefficients also identify their fitted unit or group, coefficient, allocation row, and parent context. R-hat and ESS use the selected draws within their original chains, even when storage has been pooled. One-chain selections report `NA` for R-hat.

Value

A data frame of class `magCompSummary`, identifying `component`, `block`, `term`, `parameter`, `index`, and manuscript `symbol`, followed by posterior means, standard deviations, requested quantiles, $\widehat{R}$, effective sample size, retained samples per chain, and number of chains.

See Also

Fitted and prediction summaries, magCompNotation, `plot`, `samples`

`summary.magCompPrediction`

Summarize stored fitted, prediction, or stand draws

Description

Calculates means, standard deviations, and quantiles from stored draws, pooling chains. No new observations or latent effects are simulated.

Usage

```
## S3 method for class 'magCompPrediction'
summary(object, probs = c(0.025, 0.5, 0.975), ...)
```

Arguments

`object`	Stored draws returned by `fitted()`, `predict()`, or `standSummary()` with `summary = "samples"`.
`probs`	Nonempty numeric vector of quantile probabilities strictly between zero and one. Defaults give the median and a 95% interval.
`...`	Currently unused.

Details

Expected-target intervals describe uncertainty in conditional means. Observation-target intervals also include variation from the observation model. Proportions from observation predictions or stand aggregation are undefined at zero totals: their summaries use positive-total draws only. If none remain, the summary statistics are `NA` and `n_draws` is zero. Counts, totals, densities, and magnitudes retain zero-valued draws. Standard deviations are `NA` when fewer than two draws contribute.

Objects containing only means or quantiles cannot supply these summaries. Retain draws with `summary = "samples"` in the original call. For stand results, units are stand IDs and quantities are density, total, or proportion; the same calculations apply. This method summarizes each unit and category separately; use `standSummary()` or aggregate stored draws before summarizing to obtain area-level intervals.

Value

A data frame with one row per unit, quantity, and category. Its class is `magCompPredictionSummary`. The identifying columns are `unit`, `quantity`, and `category`; magnitude uses category `"total"`. Columns `mean`, `sd`, and the requested quantiles (for example, `q2_5`, `q50`, and `q97_5`) summarize each row; `n_draws` counts the draws used. Rows follow the requested quantity order, then category order, with units in their original order within each category. The `metadata` attribute preserves the source object's metadata, including measurement units, target, and conditioning information. The `probs` attribute records the summary's quantile probabilities.

See Also

magCompPrediction, `summary` for parameter summaries, `standSummary`, `ppcSummary`

Examples

```
## Not run:
draws <- predict(fit, newdata, summary = "samples")
summary(draws)
summary(draws, probs = c(0.05, 0.5, 0.95))

replicates <- fitted(fit, target = "observation", summary = "samples")
summary(replicates)

## End(Not run)
```

Index

factor_gp *(magCompTerms)*, 28
factor_iid *(magCompTerms)*, 28
factor_nngp *(magCompTerms)*, 28
fitted, *3*, *6*, *22*, *33*, *35*, *37*, *41*
Fitted and prediction summaries, *43*
fitted.magComp, 4
fixed *(magCompCovModels)*, 10
fixedArea, *3*, *17*
fixedArea *(magCompData)*, 15
fixedAreaStems, *4, 5*, *12*, *17*, *34, 35*
fixedAreaStems *(magCompFamilies)*, 17

gp *(magCompTerms)*, 28

iid *(magCompTerms)*, 28

magComp, *3, 4*, 5, *8*, *12*, *17*, *22*, *24*, *36*
magComp-package, 2, *6–8*, *10*, *18*, *22, 23*, *26, 27*, *30*, *41, 42*
magCompComposition, *3*, *5*, 7, *7*, *12*, *17*, *26*
magCompControl, *3*, *6, 7*, 8, *11*, *18*
magCompCovModels, *6*, 10, *13*, *19*, *23*, *28, 29*
magCompCV, *3*, *6*, 12, *33*, *35*
magCompData, *3*, *5–7*, *12*, 15, *18*, *25*
magCompFamilies, 17
magCompNotation, *3, 4*, *6*, 18, *23, 24*, *28*, *30*, *38, 39*, *43*
magCompPrediction, *3–5*, *14*, 19, *33*, *35*, *40, 41*, *44*
magCompPriors, *3*, *5, 6*, *10, 11*, *17–19*, 22, *26*, *29, 30*
magCompSettings, *3*, *6*, *11*, *19*, *22, 23*, 24, *26*, *29*, *37, 38*, *43*
magCompSmooth, *3*, *5*, *8*, *23*, 25
magCompStarting, *3*, *6*, *11*, *24*, 26, *30*, *37*
magCompTerms, *3*, *5, 6*, *8*, *11*, 28

nngp *(magCompTerms)*, 28

plot, *3*, *43*
plot.magComp, 30
plot.magCompCV *(magCompCV)*, 12
plot.magCompSamples *(plot.magComp)*, 30
pointBasalArea, *4, 5*, *12*, *17*, *34, 35*
pointBasalArea *(magCompFamilies)*, 17
pointSampling, *3*, *17*
pointSampling *(magCompData)*, 15
ppcSummary, *3–6*, 31, *44*
predict, *3*, *5, 6*, *12–14*, *22*, *29*, *37*, *41*
predict.magComp, 33
print.magCompSamples *(samples)*, 35

samples, *3*, *6*, *19*, *28*, *31*, *34*, 35, *43*
spatialDiagnostics, *3*, 38
standSummary, *3*, *5, 6*, *18–20*, *35*, 39, *44*
summary, *3*, *5, 6*, *10*, *19*, *21*, *31*, *35*, *38*, *41*, *44*
summary.magComp, 42
summary.magCompPrediction, 43
summary.magCompSamples *(summary.magComp)*, 42

uniform *(magCompCovModels)*, 10